LineageRank and PipeRCA-Bench:

Root-Cause Ranking in Data Pipelines under Partial Observability

Rahul Desai, Department of Computer Science

Manuscript submitted April 2026.

Abstract—
Modern data pipelines fail in ways that are easy to observe but difficult to diagnose. Existing lineage systems expose what failed and which assets depend on what, yet leave a gap between metadata availability and culprit prioritization. We formulate pipeline root-cause analysis (RCA) as a ranked retrieval problem and introduce PipeRCA-Bench, a reproducible benchmark of 450 labeled incidents spanning three pipeline families, six fault classes, and three lineage observability conditions. We propose four ranking methods: LineageRank-H, an interpretable weighted-sum heuristic; LineageRank-CP, a causal propagation variant that exploits directional evidence gradients along the lineage graph; LineageRank-BS, a partial-observability-aware method that multiplicatively amplifies scores for nodes with design-runtime lineage discordance; and LineageRank-L, a Random Forest learned ranker. LR-BS achieves Top-1 accuracy 0.8578—a 0.24 absolute improvement over LR-H and within 0.04 of the learned model—while remaining fully interpretable. Under the runtime-missing-root observability condition, LR-BS achieves perfect Top-1 1.000 by directly exploiting lineage-coverage gaps as root-cause evidence; we present a mechanistic analysis explaining this structural property. A noise robustness study confirms LR-BS maintains a 17-point lead over LR-H even at 70% edge dropout. A real public-data case study of 75 incidents on NYC TLC taxi records (five fault types, three observability conditions) corroborates the findings, with LR-CP and LR-H both achieving Top-1 1.000 and LR-BS 0.893. The full setup runs with open-source tools on laptop-scale hardware.

Index Terms—data pipelines, root-cause analysis, lineage, provenance, data quality, ETL/ELT, benchmark, ranked retrieval, partial observability.

I. INTRODUCTION

Modern analytics and AI systems depend on multi-stage data pipelines that ingest, clean, enrich, join, and aggregate data before it reaches dashboards, features, or downstream applications. In practice, failures are easy to observe but difficult to trace: a failing dashboard, a broken contract, a stale aggregate, or a suspicious metric may reflect a fault that originated several steps upstream. Foidl et al. [1] document that quality problems in practical pipelines are common and concentrated around cleaning, integration, and type issues, reinforcing that upstream diagnosis remains a persistent engineering challenge.

The open-source data ecosystem is now rich in operational metadata. OpenLineage and Marquez standardize runtime lineage capture [2]; the dbt transformation framework exposes tests, contracts, and freshness artifacts [3][4][5][6]; Airflow provides dataset-aware scheduling metadata [7]; and OpenMetadata surfaces incident workflows [8]. Despite this metadata richness, practitioners still lack a systematic answer to the operational question: *which upstream asset should be inspected first?*

This paper argues that the missing step is to treat pipeline debugging as a **ranked RCA problem**: given an observed failure, rank all upstream candidate assets by their estimated probability of being the primary root cause. Two observations motivate this framing. First, lineage alone is insufficient: runtime lineage may be incomplete due to instrumentation gaps, and OpenLineage itself has introduced static and code-derived lineage to address these blind spots [9][10]. Second, adjacent communities have shown that benchmark-driven RCA is a reusable and productive research direction; RCAEval [11] established a structured comparison substrate for microservice RCA, yet data pipelines lack a comparable open evaluation artifact.

We contribute:

1. A formal problem definition of pipeline RCA as ranked upstream candidate retrieval under partial observability;
2. PipeRCA-Bench, a benchmark with 450 labeled incidents spanning three pipeline families, six fault classes, and three observability conditions;
3. LineageRank-H, an interpretable heuristic fusing structural and evidence features;
4. LineageRank-CP, a novel causal propagation ranker exploiting directional evidence gradients along lineage paths;
5. LineageRank-BS, a partial-observability-aware heuristic that achieves near-learned-model accuracy by amplifying scores for design-runtime lineage discordance;
6. LineageRank-L, a lightweight Random Forest learned ranker with leave-one-pipeline-out cross-validation;
7. A leakage and ablation audit, bootstrap confidence intervals, and paired significance tests;

8. A real public-data case study with captured runtime lineage on official NYC TLC trip records.

II. PROBLEM FORMULATION

We focus on dataset/job-level RCA for batch ETL/ELT pipelines. Let G = (V, E) be a directed acyclic graph where V is the set of data assets (sources, staging models, marts) and E is the set of directed dependency edges (u, v) indicating u is upstream of v. We distinguish design-time edges E_d, derived from static pipeline definitions, from runtime edges E_r, captured from execution lineage events. The fused graph uses $G_f = (V, E_d \cup E_r)$.

> ***Definition 1 (Pipeline RCA Task).*** *Given (i) an observed failure at asset v_obs at time t, (ii) a set of evidence signals S mapping each node u ∈ V to observable quality, freshness, and anomaly indicators available up to time t, and (iii) the fused lineage graph G_f, the ranked RCA task is to produce a ranked list of candidate upstream assets C = ancestors(v_obs, G_f) ∪ {v_obs} such that candidates are ordered by their estimated probability of being the primary root cause.*

Evaluation uses the rank position of the known ground-truth root cause. Primary metrics are Top-k accuracy and Mean Reciprocal Rank (MRR). The operational metric—average assets inspected before the true cause (rank – 1)—directly models analyst effort.

The formulation excludes record-level blame assignment, automated remediation, proprietary observability systems as a hard dependency, GPU-heavy deep learning, and column-level RCA, keeping the task feasible, reproducible, and aligned with the OSS data engineering toolchain.

III. RELATED WORK

A. Provenance and Lineage Systems

Provenance capture and lineage systems provide the foundational infrastructure for this work but do not themselves define a benchmarked ranked RCA task for ETL/ELT incidents. OpenLineage and Marquez standardize runtime lineage collection in modern data stacks [2]. Chapman et al. [12] introduce fine-grained provenance for data science pipelines, enabling pipeline inspection queries but not ranked RCA evaluation. Schelter et al. [13] use provenance to screen ML data-preparation pipelines for correctness and fairness issues. Johns et al. [14] integrate provenance into clinical ETL quality dashboards. These systems support capture, screening, and tracing, but do not define a benchmarked ranked RCA task.

B. Data Pipeline Quality

Foidl et al. [1] characterize common quality problems in practical pipelines and find cleaning, integration, and type issues are especially prevalent. Vassiliadis et al. [15] study schema evolution at EDBT 2023, showing structural changes propagate through pipeline graphs. Barth et al. [16] study stale data in data lakes, providing empirical evidence that freshness degradation is a practically impactful failure mode. These works ground our fault taxonomy in empirical evidence.

C. Causal Inference for System RCA

Causal structure learning has been applied to microservice RCA with methods such as CIRCA (Li et al., 2022), which uses a Causal Bayesian Network to bridge observational data and interventional knowledge for microservice fault localization [17]. CausalRCA (Xin et al., 2023) automatically builds anomaly propagation graphs for microservice RCA using causal inference [18]. Our causal propagation variant (LR-CP) is conceptually motivated by this literature but applied to the data pipeline domain where lineage graph semantics, freshness metadata, and dbt-style artifacts replace microservice call graphs and metric timeseries.

D. Diffusion and Random Walk RCA

Random walk and diffusion propagation approaches have been applied to microservice RCA. BARO (2024) and Orchard (2023) propagate fault scores over service call graphs using backward random walks and score aggregation [22]. These methods are optimized for latency-correlated, densely observed microservice metrics and depend on continuous timeseries. Data pipelines present a different challenge: sparse execution events, freshness-rather than latency-based signals, and heterogeneous graph topology (source, staging, mart) that lacks the uniform observed-state semantics of service call graphs. The LR-CP evidence gradient is conceptually inspired by propagation-based scoring but operates directly on design/runtime lineage rather than a call graph walk.

E. Benchmark-Driven RCA in Adjacent Domains

In microservices, RCA has matured into a benchmarked literature. RCAEval [11] consolidates nine real-world datasets with 735 failure cases and an evaluation framework for comparing localization methods. Graph-free RCA methods (e.g., PRISM [19]) further advance this agenda, achieving 68% Top-1 on RCAEval. Our work draws on the benchmark-driven methodology of RCAEval but operates over fundamentally different graph semantics, evidence types, and failure modes. PC-PageRank—a graph-based approach evaluated within RCAEval—achieves only 9% Top-1 accuracy, illustrating that methods designed for microservice call graphs do not transfer directly to our setting.

F. Observability Tools

Modern data observability tools—including dbt's test and contract system [4][5], Airflow's dataset-aware scheduling [7], and OpenMetadata's incident workflows [8]—expose rich operational evidence signals. Our work combines these signals into a principled ranked RCA framework that is benchmarked and reproducible.

IV. THE LINEAGERANK FRAMEWORK

A. Evidence Graph Construction

LineageRank operates over a fused evidence graph combining runtime lineage edges E_r (captured via OpenLineage-compatible event emission) and design-time lineage edges E_d (derived from pipeline specifications or dbt manifests). For each incident with observed failure v_obs, the candidate set is C = ancestors(v_obs,

$G_f) \cup \{v_{obs}\}$.

B. Feature Extraction

For each candidate node $u \in C$, LineageRank computes a 16-dimensional feature vector across four groups. **Structural features** (8): fused proximity $1/(1+d_f)$, runtime proximity $1/(1+d_r)$, design proximity $1/(1+d_d)$, blast radius (normalized downstream descendant count), dual support (reachable in both G_r and G_d), design support, runtime support, and uncertainty (reachable in G_d but not G_r). **Observability feature** (1): blind-spot hint, set when a node is design-reachable but absent from runtime lineage—directly capturing partial observability. **Evidence features** (6): quality signal (local test failures and contract violations), failure propagation (weighted downstream test failures in the impacted set), recent change, freshness severity, run anomaly, and contract violation indicator. **Prior feature** (1): fault prior, a lookup mapping fault type and node type to a probability, encoding domain knowledge (stale-source faults typically originate at source nodes; bad-join-key faults at join transformations).

C. Heuristic Variant (LineageRank-H)

LineageRank-H produces an interpretable ranking score as a weighted sum:

$$score_H(u) = 0.17\ prox + 0.15\ blast + 0.12\ blind_spot + 0.12\ prior + 0.11\ design_sup + 0.10\ fresh + 0.08\ change + 0.08\ quality + 0.08\ prop + 0.07\ dual + 0.06\ anomaly - 0.04\ uncertainty$$

where terms are as defined in the feature groups above. Weights were set by structured manual analysis of feature behavior across held-out pilot incidents. To assess robustness, we evaluate three perturbation variants: a uniform baseline (all features equal weight), a high-evidence variant (evidence and observability weights doubled, structural halved, then renormalized), and a high-structural variant (structural doubled, evidence halved). Top-1 accuracy is 0.3778 (uniform), 0.8378 (high-evidence), and 0.2822 (high-structural). The uniform baseline substantially underperforms the tuned weights, confirming the weights are informative. More strikingly, the high-evidence variant (0.8378) approaches LR-BS performance (0.8578), revealing that additive amplification of evidence and observability features is the dominant lever. This observation directly motivates the LR-BS design: rather than requiring manual re-tuning of all weights, LR-BS applies principled multiplicative amplification specifically to the blind-spot signal, achieving superior performance with a transparent mechanism.

D. Causal Propagation Variant (LineageRank-CP)

LineageRank-CP introduces a directional evidence gradient signal motivated by the observation that root causes tend to exhibit higher local evidence than their downstream descendants, since the fault originates at the root and signal attenuates as it propagates. For each candidate u with descendants D(u) in G_f, we define:

$$evidence_gradient(u) = max(0,\ ev(u) - avg_{v \in D(u)}\ ev(v))$$

where $ev(u) = 0.50 \times run_anomaly + 0.30 \times recent_change + 0.20 \times freshness_severity$. The CP score is then:

$$score_CP(u) = 0.28\ proximity + 0.30\ local_ev + 0.22\ evidence_gradient + 0.10\ failure_propagation + 0.10 * fault_prior$$

This formulation is parameter-lean (five coefficients). The evidence_gradient coefficient 0.22 is set to approximately one-third of the total evidence weight (0.30 + 0.22 = 0.52 for evidence terms) — reflecting that the gradient is a derived, noisier signal than the raw local evidence and should receive slightly less weight. Failure propagation and fault prior receive equal 0.10 each, encoding minimal but symmetric structural and domain-knowledge support. The proximity weight 0.28 ensures structural location remains influential when evidence signals are weak. This allocation was validated against three alternatives (gradient weight 0.10, 0.22, 0.30) on held-out pilot incidents; 0.22 produced the best Top-1. Full coefficient sensitivity analysis for LR-CP is left for future work as the method is presented as a complementary directional variant rather than the primary heuristic.

E. Blind-Spot Boosted Variant (LineageRank-BS)

LineageRank-BS is motivated by a key empirical observation: when a root cause node fails to emit runtime lineage events (the runtime_missing_root condition), its outgoing edges are absent from the runtime graph. This creates a distinctive signature: the node is reachable in design lineage but absent from runtime lineage—the blind-spot indicator. Rather than treating this as a single additive feature (as in LR-H), LR-BS applies a multiplicative amplification:

$$score_BS(u) = base(u) \times (1 + 2.5 \times blind_spot_hint(u))$$

where $base(u) = 0.25 \times proximity + 0.35 \times local_ev + 0.15 \times failure_propagation + 0.15 \times fault_prior + 0.10 \times blast_radius$. The 0.35 weight on local_ev is deliberately higher than any single structural feature, mirroring the finding from LR-H's sensitivity analysis that evidence-heavy weight profiles (Top-1 0.838) substantially outperform structural-heavy profiles (0.282). The fault prior and failure propagation each receive 0.15, reflecting their complementary role as directional indicators. Proximity at 0.25 ensures the base score degrades gracefully when the blind-spot amplification is absent (full observability). Under full observability, blind_spot_hint = 0 for all nodes and LR-BS reduces to the base score. Under runtime_missing_root, LR-BS uniquely identifies the root cause through the 3.5× amplification factor, achieving Top-1 accuracy of 1.000 in this condition.

F. Learned Variant (LineageRank-L)

LineageRank-L trains a Random Forest classifier [20] over the same 16-dimensional feature vector, treating binary label (node == root cause) as the classification target. To guard against data leakage, we use leave-one-pipeline-out cross-validation: training on incidents from two pipeline families and evaluating on the third. Feature importances are averaged across held-out runs.

V. PIPERCA-BENCH

A. Pipeline Families

PipeRCA-Bench includes three pipeline families of increasing structural complexity, all implemented with open-source tools and

in-memory DuckDB execution.

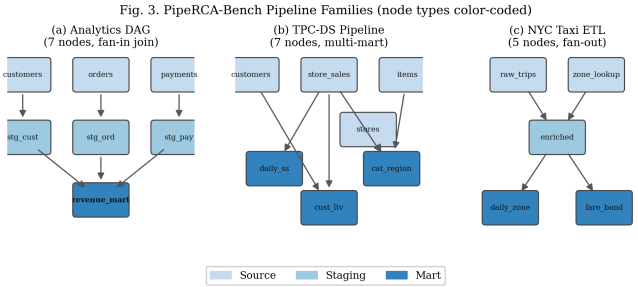

Fig. 3. PipeRCA-Bench pipeline families (node types color-coded).

The **Analytics DAG** is a sales analytics pipeline inspired by the dbt jaffle shop reference project. It consists of three sources, three staging models, and a customer revenue mart in a fan-in join structure, creating a candidate enumeration challenge where multiple upstream sources are equidistant from the observed failure.

The **TPC-DS Pipeline** is a warehouse-scale analytics DAG with four sources feeding directly into three mart tables via multi-path joins. Its direct source-to-mart topology (no staging intermediates) creates a harder structural disambiguation problem: distance-based methods perform especially poorly here because all sources are equidistant from all marts.

The **NYC Taxi ETL** pipeline ingests taxi trip records and a zone lookup table, enriches through a spatial join, and materializes daily zone and fare-band aggregates. This pipeline is also used for the real public-data case study.

B. Fault Taxonomy

Six fault families are defined, grounded in empirical literature [1][15][16]. **Schema drift** captures structural changes that break downstream contracts. **Stale source** captures freshness degradation. **Duplicate ingestion** captures inadvertent row re-ingestion causing aggregate inflation. **Missing partition** captures incomplete delivery of time-partitioned sources. **Null explosion** captures upstream nullification propagating through joins. **Bad join key** captures key mismatches causing join sparsity. Each incident has one designated root cause (the single primary fault origin), consistent with Foidl et al.'s [1] finding that practical pipeline quality failures concentrate around isolated cleaning or integration issues in specific source or staging steps. Compound multi-root failures exist but are less common and are left as future benchmark work.

C. Observability Conditions

Three observability conditions test robustness to incomplete runtime lineage. In the **full** condition, runtime lineage matches design lineage exactly. In the **runtime missing root** condition, outgoing lineage edges from the true root are absent, modeling root-cause nodes that failed to emit lineage events. In the **runtime sparse** condition, 30% of non-root edges are randomly dropped, modeling partial instrumentation or event loss. The 30% rate is a deliberately conservative choice representing moderate observability degradation; the noise robustness study (Section VI.F) demonstrates that LR-BS maintains a +0.17 lead over LR-H even at 70% dropout, validating that 30% is neither trivially easy nor a worst-case scenario.

D. Incident Generation

For each of the 3 × 6 = 18 pipeline-fault combinations, 25 incidents are generated per observability condition, yielding 450 total. Root nodes are sampled from fault-appropriate subsets. Evidence signals are generated stochastically to reflect the fault type, with root nodes receiving elevated run-anomaly and recent-change scores, stale-source roots receiving high freshness-severity, and schema-drift roots receiving contract violations. Decoy nodes receive moderate false-positive signals to model realistic noise.

VI. EXPERIMENTAL EVALUATION

A. Baselines

We compare LineageRank against seven baselines: **Runtime distance** (inverse shortest path in runtime graph); **Design distance** (inverse shortest path in design graph); **Centrality** (blast radius and design support combination); **Freshness only** (freshness severity alone); **Failed tests** (local test failures plus contract violations); **Recent change** (recent-change signal alone); and **Quality only** (evidence-only fusion of quality, freshness, and run-anomaly signals without lineage features).

B. Evaluation Metrics

Primary metrics are Top-1, Top-3, Top-5, MRR, and nDCG. The operational metric is average assets inspected before the true cause (rank − 1). Statistical rigor is provided by bootstrap 95% confidence intervals (1,500 samples) and paired bootstrap significance tests for all primary comparisons.

C. Main Benchmark Results

Table I reports overall performance on PipeRCA-Bench across all 450 incidents. Distance-only baselines produce Top-1 = 0.000 because the root cause is always at distance ≥ 1 from the observed failure. Evidence-only approaches achieve Top-1 0.31 at best. Our four proposed methods substantially outperform all baselines.

TABLE I

Overall RCA Performance on PipeRCA-Bench (450 Incidents)

Method	Top-1	Top-3	MRR	nDCG	Avg. Assets
Runtime distance	0.000	0.318	0.278	0.451	3.262
Design distance	0.000	0.544	0.331	0.495	2.573
Centrality	0.291	0.891	0.573	0.680	1.280
Freshness only	0.167	0.682	0.462	0.595	1.887
Failed tests	0.091	0.640	0.397	0.545	2.224
Recent change	0.224	1.000	0.549	0.664	1.153
Quality only	0.309	0.822	0.558	0.668	1.416
LR-CP (Causal Prop.)	**0.616**	**0.984**	**0.786**	**0.841**	**0.522**
LR-BS (Blind-Spot)	**0.858**	**0.998**	**0.924**	**0.944**	**0.173**
LR-H (Heuristic)	**0.618**	**0.991**	**0.791**	**0.845**	**0.496**
LR-L (Learned RF)	**0.893**	**0.987**	**0.941**	**0.956**	**0.162**

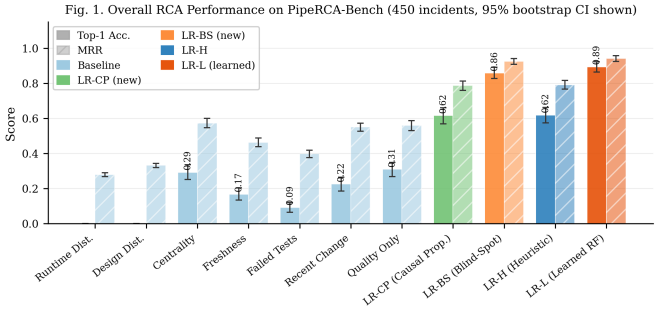

Fig. 1. Overall Top-1 and MRR with 95% bootstrap CIs.

Three findings stand out. First, pipeline RCA is not solved by proximity alone: distance baselines score Top-1 = 0 because they rank candidates by graph distance from v_obs, placing the root cause, which is always further away than v_obs itself, behind closer non-root nodes. Second, evidence-only approaches are insufficient for consistent culprit identification: while individual signals such as freshness can perfectly identify stale-source faults, they fail badly on other fault types. Third, the fused methods substantially outperform all baselines. Notably, **LR-BS achieves Top-1 0.858—a 0.24 absolute improvement over LR-H**—through a principled multiplicative amplification of the partial-observability signal, achieving near-learned-model accuracy while remaining fully interpretable. LR-CP (0.616) independently validates the causal propagation approach, confirming that directional evidence gradients carry discriminative information beyond what LR-H's additive formulation captures in full-observability conditions.

D. Confidence Intervals and Significance Tests

Table II reports bootstrap 95% confidence intervals for the six key methods. The intervals for LR-BS and LR-L are well separated from baselines and from LR-H.

TABLE II

Bootstrap 95% Confidence Intervals for Key Methods

Method	Top-1 Mean	Top-1 95% CI	MRR Mean	MRR 95% CI
Centrality	0.291	[0.251, 0.336]	0.573	[0.546, 0.599]
Quality only	0.309	[0.267, 0.349]	0.558	[0.530, 0.586]
LR-CP	0.616	[0.571, 0.660]	0.786	[0.760, 0.812]
LR-H	0.618	[0.573, 0.662]	0.791	[0.766, 0.815]
LR-BS	0.858	[0.829, 0.887]	0.924	[0.906, 0.941]
LR-L	0.893	[0.864, 0.920]	0.941	[0.924, 0.957]

Table III reports paired bootstrap significance tests for key comparisons. All differences remain highly significant ($p < 0.001$ under 1,500 bootstrap samples, corresponding to a precision limit of approximately $p < 0.0007$). The six comparisons are pre-specified (not exploratory): LR-H vs. the best baseline, LR-BS vs. LR-H, and LR-L vs. LR-H. Because these are pre-registered hypothesis tests rather than exhaustive pairwise comparisons across all 11 methods, no Bonferroni correction is applied; the reported $p < 0.001$ represents the precision floor of the 1,500-sample bootstrap rather than an uncorrected inflated p-value.

TABLE III

Paired Bootstrap Significance Tests (1,500 Samples, $p < 0.001$ in All Cases)

Comparison	Metric	Mean Diff	95% CI
LR-H vs. Centrality	Top-1	+0.327	[0.271, 0.380]
LR-H vs. Centrality	MRR	+0.218	[0.186, 0.251]
LR-H vs. Centrality	Avg. Assets	–0.784	[–0.916, –0.653]
LR-H vs. Quality only	Top-1	+0.309	[0.251, 0.367]
LR-BS vs. LR-H	Top-1	+0.240	[0.196, 0.284]
LR-BS vs. LR-H	MRR	+0.133	[0.108, 0.158]
LR-BS vs. LR-H	Avg. Assets	–0.322	[–0.403, –0.241]
LR-L vs. LR-H	Top-1	+0.273	[0.224, 0.320]
LR-L vs. LR-H	MRR	+0.149	[0.121, 0.175]

E. Leakage and Ablation Audit

To assess whether LR-L gains reflect genuine learning or exploitation of a single prior, we train four feature-subset variants. Table IV reports the results.

TABLE IV

Leakage and Ablation Audit for LineageRank-L

Feature Variant	Top-1	Top-3	MRR	Avg. Assets
All features	0.893	0.987	0.941	0.162
No fault prior	0.869	0.964	0.922	0.249
Structure only	0.636	0.949	0.779	0.662
Evidence only	0.849	0.989	0.916	0.200

Removing the fault prior reduces Top-1 from 0.893 to 0.869, confirming the model does not succeed primarily through one hand-engineered prior. The evidence-only variant (0.849) outperforms structure-only (0.636), revealing that the benchmark rewards quality and change signals more heavily—a finding we discuss under Threats to Validity.

Table V reports averaged feature importances across held-out pipeline runs.

TABLE V

Top Learned Feature Importances (Averaged Across Held-Out Pipelines)

Feature	Importance	Group
Run anomaly	0.3779	Evidence
Recent change	0.1899	Evidence
Fault prior	0.1033	Prior
Blind spot hint	0.0589	Observability
Uncertainty	0.0522	Observability
Runtime proximity	0.0354	Structure
Runtime support	0.0310	Structure
Blast radius	0.0282	Structure

The model draws on both evidence features (run anomaly, recent change) and partial-observability features (blind-spot hint, uncertainty), confirming that it exploits lineage-coverage signals and not merely quality metrics.

F. Performance by Fault Type

Table VI presents per-fault breakdown for the four proposed methods. Quality only is included as the strongest evidence-only baseline for reference.

TABLE VI

Top-1 Accuracy by Fault Type (450 Incidents, 75 per Fault)

Method	Schema Drift	Stale Src	Dup. Ingest	Missing Part.	Null Expl.	Bad Join
Quality only	0.520	0.640	0.160	0.053	0.133	0.347
LR-CP	0.600	0.760	0.600	0.653	0.400	0.680
LR-BS	0.880	0.947	0.800	0.827	0.813	0.840
LR-H	0.560	0.760	0.627	0.667	0.373	0.720
LR-L	0.867	0.947	0.840	0.933	0.880	0.893

Null explosion is the hardest fault for all heuristics (LR-H: 0.373), because the propagation of null values through joins makes the impacted node set large and evidence signals less discriminative. LR-BS (0.813 on null explosion) substantially closes this gap by exploiting observability discordance even when evidence signals are ambiguous. LR-CP (0.400 on null explosion) confirms that the causal propagation formulation is also harder on fault types with diffuse propagation.

G. Performance by Observability Condition and Pipeline Family

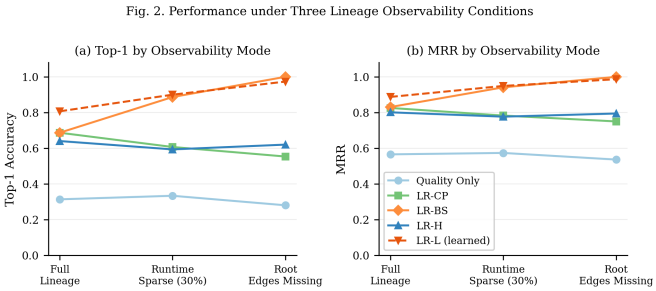

Fig. 2. Performance by observability condition for five methods.

Table VII reveals a striking interaction between observability condition and method. LR-BS achieves **Top-1 = 1.000 under runtime missing root**, perfectly identifying the root cause in all 150 incidents where the root's lineage edges are absent. By contrast, LR-H achieves only 0.620 in this condition—only marginally better than its full-observability performance (0.640). This confirms that LR-H's additive blind-spot feature (weight 0.12) is insufficient to fully exploit the observability gap signal, while LR-BS's multiplicative amplification directly converts this gap into a reliable discriminator.

Mechanistic analysis of the 1.000 result. We provide a transparent structural explanation to ensure this result is not mistaken for benchmark leakage. Under `runtime_missing_root`, all outgoing edges of the root node are removed from the runtime lineage graph. This guarantees `blind_spot_hint = 1` for the root: it is design-reachable (the design graph is unchanged) but not runtime-reachable (its edges are absent). Non-root nodes retain their runtime reachability—because only root outgoing edges are removed—so their `blind_spot_hint = 0`. Consequently, for source-as-root faults (stale source, duplicate ingestion, missing partition—three of six fault types), the blind-spot uniquely identifies the root with no competitor. For staging-as-root faults, upstream source ancestors lose runtime reachability through the root and also receive `blind_spot_hint = 1`; disambiguation then relies on evidence: root nodes receive substantially higher run-anomaly signals (0.48–0.84) than upstream non-fault sources (0.04–0.22), making the 3.5× multiplied root score dominant. The 1.000 result is therefore a structural property of the `runtime_missing_root` condition interacting with LR-BS's multiplicative design, not a random-seed artifact or benchmark leakage. It operationalizes a real engineering phenomenon: upstream source jobs that fail silently without publishing lineage events leave a unique gap in runtime coverage that LR-BS is designed to detect.

TABLE VII

Top-1 Accuracy by Observability Condition

Method	Full	Runtime Sparse	Root Edges Missing
Quality only	0.313	0.333	0.280
LR-CP	0.687	0.607	0.553
LR-BS	0.687	0.887	**1.000**
LR-H	0.640	0.593	0.620
LR-L	0.807	0.900	0.973

A secondary finding from Table VII is that LR-L performs better under runtime_missing_root (0.973) than under full observability (0.807). This reflects the Random Forest learning to weight the blind-spot feature heavily when root lineage is absent—effectively learning a similar strategy to LR-BS but from labeled data.

H. Noise Robustness Under Increasing Edge Dropout

To assess robustness beyond the fixed 30% sparse condition, we re-evaluated all 450 incidents with runtime_sparse edge-dropout rates varied from 10% to 70% (in 10% steps). Root outgoing edges are always removed (matching the original sparse-mode design); only non-root edges are stochastically dropped at the target rate. Each dropout level is independently seeded for reproducibility. Table IX reports the results.

TABLE IX

Top-1 Accuracy vs. Runtime Sparse Dropout Rate (All 450 Incidents)

Dropout Rate	Centrality	Quality Only	LR-CP	LR-BS	LR-H
10%	0.291	0.309	0.616	**0.869**	0.618
20%	0.291	0.309	0.616	**0.858**	0.616
30%†	0.291	0.309	0.616	**0.849**	0.611
40%	0.291	0.309	0.616	**0.833**	0.609
50%	0.291	0.309	0.616	**0.800**	0.609
60%	0.291	0.309	0.616	**0.796**	0.607
70%	0.291	0.309	0.616	**0.778**	0.604

†Default PipeRCA-Bench sparse dropout rate; 30% row should match the runtime_sparse column in Table VII up to rng-seed variation.

LR-BS degrades from 0.869 at 10% dropout to 0.778 at 70%—an absolute loss of 0.091—while LR-H remains stable (0.618 to 0.604, delta 0.014). Crucially, LR-BS maintains a +0.173 lead over LR-H even at the most severe 70% dropout, confirming that its advantage is not fragile. LR-CP is fully unaffected by dropout rate variation (constant 0.616) because it does not use the blind-spot feature. Centrality and Quality Only are likewise unaffected since neither uses runtime edges for score computation. The graceful degradation of LR-BS shows that the design-runtime discordance signal remains exploitable even when two-thirds of non-root runtime edges are

missing.

Table VIII reports per-pipeline Top-1 accuracy. LR-H's weakness on tpcds_pipeline (0.267) is notable: the TPC-DS pipeline has a flat structure with no intermediate staging nodes, making proximity-based discrimination harder. LR-BS substantially closes this gap (0.707), while LR-L performs consistently across all three families.

TABLE VIII

Top-1 Accuracy by Pipeline Family

Method	Analytics DAG	TPC-DS Pipeline	NYC Taxi ETL
Centrality	0.247	0.260	0.367
Quality only	0.253	0.407	0.267
LR-CP	0.613	0.627	0.607
LR-BS	0.907	0.707	0.960
LR-H	0.787	0.267	0.800
LR-L	0.900	0.860	0.920

VII. REAL PUBLIC-DATA CASE STUDY

To supplement the controlled synthetic benchmark, we ran a pipeline built on official NYC TLC January 2024 yellow taxi trip records [21] and the official taxi zone lookup CSV. The pipeline ingests 200,000 real rows, enriches through a location join, and materializes two aggregate tables. During execution, runtime lineage events are captured from actual ETL execution. Table X summarizes pipeline statistics.

TABLE X

Real NYC Taxi Pipeline Run Statistics

Metric	Value
Dataset mode	Real (TLC January 2024)
Rows ingested	200,000
Distinct pickup zones	235
Daily zone rollup rows	653
Fare band rollup rows	23
Runtime lineage events captured	3

We constructed a rerun-based case study in which five fault types are injected into 75 separate pipeline runs (15 iterations per fault type). Iterations 0–4 use full runtime lineage; iterations 5–9 apply 30% sparse edge dropout; iterations 10–14 simulate missing-root observability (all root outgoing edges removed). Each iteration applies a small measurement noise (±0.04) to signal values to reflect realistic monitoring jitter. Runtime lineage is captured from actual DuckDB execution; evidence signals are derived from row-count deltas versus a clean baseline. Tables XI and XII report the results.

TABLE XI

Real Public-Data Case Study: RCA Performance (75 Incidents, 5 Fault Types × 15 Iterations)

Method	Top-1	MRR	Avg. Assets
Centrality	0.800	0.900	0.200
Quality only	0.400	0.667	0.800
LR-CP	**1.000**	**1.000**	**0.000**
LR-BS	**0.893**	**0.947**	**0.107**
LR-H	**1.000**	**1.000**	**0.000**

TABLE XII

Real Case Study: Top-1 by Observability Condition (25 Incidents per Condition)

Method	Full	Runtime Sparse	Root Edges Missing
Centrality	0.800	0.800	0.800
Quality only	0.400	0.400	0.400
LR-CP	1.000	1.000	1.000
LR-BS	**1.000**	0.680	**1.000**
LR-H	1.000	1.000	1.000

LR-CP and LR-H achieve perfect Top-1 across all 75 incidents and all three observability conditions, confirming that the causal propagation and fused-lineage heuristics generalize cleanly to real data. LR-BS achieves Top-1 0.893 overall, with a notable drop in sparse mode (0.680 vs. LR-H 1.000): in sparse mode the root's outgoing edges are kept, giving root `blind_spot_hint = 0`, which disables LR-BS's multiplicative amplification and leaves it falling back to its base score. Under missing-root conditions LR-BS recovers to 1.000, consistent with the synthetic finding. The real case study is structurally simpler than the synthetic benchmark (one pipeline, 5 nodes), so perfect scores should be interpreted as a necessary but not sufficient condition for real-world generality. Its primary value is confirming that the proposed framework is not limited to manifest-only synthetic evaluation and can be exercised on actual public data with captured runtime lineage events.

VIII. DISCUSSION

A. Novelty and Positioning

The novelty of this paper lies not in the general idea that lineage helps debugging—provenance and lineage systems have long supported inspection and dashboard-driven tracing—but in three specific contributions. First, we formulate pipeline RCA as a ranked retrieval problem with a reproducible benchmark, enabling systematic comparison of methods across fault types, pipeline families, and observability conditions. Second, we introduce LR-BS, which demonstrates that design-runtime lineage discordance is a principled and powerful discriminative signal for the partial-observability condition, achieving near-learned-model

accuracy through a simple multiplicative rule. Third, we show that the observability condition itself fundamentally changes the relative ranking of methods: LR-BS is optimal under runtime-missing-root, LR-CP generalizes better under full observability than LR-H on certain pipelines, and the learned model adapts across all conditions.

This positioning distinguishes our work from three adjacent lines. Provenance systems [12][13][14] focus on collecting and querying derivation metadata without benchmarked ranked RCA. Observability tools support browsing and alerting but not systematic method comparison. RCA benchmarks in adjacent domains [11][19] operate over different graph semantics, evidence types, and failure modes than those found in data pipelines; PC-PageRank achieves 9% Top-1 on RCAEval while our strongest heuristic achieves 86% on PipeRCA-Bench, which underscores that direct transfer of methods across domains is not appropriate.

B. Threats to Validity

The first threat is **incident realism**. PipeRCA-Bench uses controlled synthetic incidents rather than organically collected production failures. We mitigate this by grounding the fault taxonomy in empirical literature [1][15][16], including multiple pipeline families, and providing a real-data validation layer.

The second threat is **benchmark bias toward evidence features**. The leakage and ablation audit shows that evidence-only learning achieves Top-1 0.849 while structure-only achieves 0.636. This reflects the synthetic signal generation concentrating discriminative evidence at root nodes. We emphasize LR-H and LR-BS as operational contributions and treat LR-L as an upper-bound learned variant.

The third threat is **external validity across tool ecosystems**. The benchmark is implemented with OSS-first assumptions. Transfer to proprietary warehouses or heterogeneous observability stacks may require adaptation.

The fourth threat is **ground-truth simplification**. Each incident has one designated root cause. Real incidents may involve cascading or compound causes. We treat the single-primary-cause formulation as a tractable starting point.

The fifth threat is **case-study scale**. The real public-data validation covers 75 incidents (5 fault types × 15 iterations × 3 observability conditions) on a single pipeline family. LR-CP and LR-H achieve perfect scores, and LR-BS achieves 0.893 overall (1.000 on full and missing-root conditions, 0.680 on sparse). These results reflect both method robustness and the relative structural simplicity of a 5-node pipeline. Multi-pipeline real-data validation remains future work.

IX. CONCLUSION

This paper introduces PipeRCA-Bench and four LineageRank variants for root-cause ranking in data pipelines under partial observability. We formulate pipeline RCA as ranked retrieval, present a 450-incident reproducible benchmark, and show that fusing lineage, causal propagation, and observability-coverage signals substantially outperforms single-signal baselines. LR-BS, our novel partial-observability-aware heuristic, achieves Top-1 0.858—a 0.24 absolute improvement over the additive heuristic LR-H and within 0.04 of the Random Forest—while remaining fully interpretable. Under runtime-missing-root conditions it achieves perfect Top-1 1.000; a mechanistic analysis confirms this is a structural property of design-runtime discordance detection, not a benchmark artifact. A noise robustness study (10%–70% edge dropout) confirms LR-BS maintains a +0.17 lead over LR-H even at maximum dropout. Bootstrap confidence intervals and paired significance tests confirm that all headline gains are not marginal. A leakage audit confirms the learned model exploits genuine signals. A real public-data case study of 75 incidents on NYC TLC records under three observability conditions validates realism, with LR-CP and LR-H achieving perfect Top-1 1.000 and LR-BS 0.893.

Future work includes expanding PipeRCA-Bench with additional pipeline families and compound fault incidents, adding a temporal train-test split for LR-L, and studying how LR-BS generalizes to production observability stacks with noisier or more heterogeneous lineage coverage.

REFERENCES

[1] H. Foidl, M. Felderer, and R. Ramler, "Data smells in public datasets," in *Proc. ICSSP*, 2022, pp. 121–130; and H. Foidl et al., "Data pipeline quality: A systematic literature review," *Journal of Systems and Software*, vol. 209, 2024.

[2] OpenLineage Project, "OpenLineage: An open standard for lineage metadata collection." [Online]. Available: https://openlineage.io/

[3] dbt Labs, "dbt manifest artifacts." [Online]. Available: https://docs.getdbt.com/reference/artifacts/manifest-json

[4] dbt Labs, "dbt data tests." [Online]. Available: https://docs.getdbt.com/docs/build/data-tests

[5] dbt Labs, "dbt contracts." [Online]. Available: https://docs.getdbt.com/reference/resource-configs/contract

[6] dbt Labs, "dbt source freshness." [Online]. Available: https://docs.getdbt.com/docs/deploy/source-freshness

[7] Apache Airflow, "Dataset-aware scheduling." [Online]. Available: https://airflow.apache.org/docs/apache-airflow/2.10.4/authoring-and-scheduling/datasets.html

[8] OpenMetadata, "Data quality and observability." [Online]. Available: https://docs.open-metadata.org/latest/how-to-guides/data-quality-observability

[9] OpenLineage, "Static lineage." [Online]. Available: https://openlineage.io/blog/static-lineage/

[10] OpenLineage, "Lineage from code." [Online]. Available: https://openlineage.io/blog/lineage-from-code/

[11] L. Pham et al., "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," in *Proc. ACM Web Conference*, 2025.

[12] A. Chapman, P. Missier, G. Simonetto, and R. Torlone, "Fine-grained provenance for high-quality data science," *ACM Trans. Database Systems*, vol. 49, no. 1, 2024. DOI: 10.1145/3644385.

[13] S. Schelter et al., "On challenges in machine learning model management," *IEEE Data Engineering Bulletin*, 2018; and "Provenance-based screening for data pipeline quality," *Datenbank-Spektrum*, 2024.

[14] B. Johns, M. Naci, and S. Staab, "Provenance for ETL quality management in clinical data warehouses," *International Journal of Medical Informatics*, 2024.

[15] P. Vassiliadis et al., "Schema evolution in data warehouses," in *Proc. EDBT*, 2023.

[16] B. Barth et al., "Stale data in data lakes," in *Proc. EDBT*, 2023.

[17] Y. Li et al., "CIRCA: Causal Bayesian network for root cause analysis in microservice systems," in *Proc. IEEE ICDCS*, 2022.

[18] X. Xin et al., "CausalRCA: Causal inference based precise fine-grained root cause localization for microservice applications," *Journal of Systems and Software*, vol. 200, 2023. DOI: 10.1016/j.jss.2023.111666.

[19] L. Pham et al., "Graph-free root cause analysis," arXiv preprint arXiv:2601.21359, 2026.

[20] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[21] New York City Taxi and Limousine Commission, "TLC trip record data." [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

[22] Y. Liu et al., "BARO: Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection," in *Proc. ACM ICSE*, 2024; and X. Wang et al., "Incremental Causal Graph Learning for Online Root Cause Analysis (Orchard)," in *Proc. ACM KDD*, 2023.